

Master d'informatique - 1
Architecture des Systèmes Intégrés et Optimisations
Décembre 2010 – Documents autorisés – Durée 2h

Utiliser impérativement les feuilles quadrillées fournies pour les schémas

Les réponses non justifiées seront considérées comme fausses

The aim of this exercise is to compare the execution performance of an image processing algorithm on two different Mips implementations. The first is the classical 5-stage pipeline implementation presented in class. This implementation is called **Mips**. The second, called **SS2**, is a two-pipeline superscalar realization presented in the class. **SS2** has the same clock frequency as **Mips**.

An image is made up of a set of pixels organized as an array. In a color image, each pixel is a combination of 3 colors : red, green and blue. A color pixel is therefore coded on an integer (4 bytes). The most significant byte is not used. Each of the other 3 bytes represents a color component of the pixel.

An image processing function consists in applying a certain algorithm to an array of pixels. In this exercise, we study a function that separates the components of an image to obtain 3 images: one image for the red component, one for the green and one for the blue. The images obtained by this decomposition are therefore monochrome images in which each pixel is coded on 1 byte.

```
void DecomposeImage (int  *img      , char *red_img,
                    char *grn_img, char *blu_img,
                    int  size      )
{
    unsigned int i;
    for (i=0 ; i!=size ; i++)
    {
        red_img [i] = img [i] >> 16;
        grn_img [i] = img [i] >>  8;
        blu_img [i] = img [i]      ;
    }
}
```

The code for the main loop of this function obtained from a Mips pipeline assembler is as follows.

```
Loop :          Lw      r10, 0 (r4 )
                Srl     r11, r10, 8
                Srl     r12, r10, 16

                Sb      r12, 0 (r5 )
                Sb      r11, 0 (r6 )
                Sb      r10, 0 (r7 )

                Addiu   r5 , r5 , 1
                Addiu   r6 , r6 , 1
                Addiu   r7 , r7 , 1
                Addiu   r4 , r4 , 4
                Bne     r4 , r8 , Loop
                Nop
```

At the loop entry, the register R4 contains the address of the first pixel of the input color image **img**, R5 the address of the red image, R6 the address of the green image and R7 the address of the blue image. R8 contains the end address of the input color image.

- 1- Analyze the execution of the loop on the **Mips** implementation. You are not asked to draw a diagram, but simply to indicate the data dependencies and the stall cycles on the code. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 2- Analyze the execution of the loop using a simplified diagram on the **SS2** realization. Assume that the **Loop** label is placed on an aligned address and that the instruction buffer is empty at the entry of the loop. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 3- For the **Mips** implementation, optimize the loop code using the "software pipeline" technique, clearly indicating the number of stages and which instructions belong to which stage. Then reorder to avoid lost cycles as far as possible. Calculate the average number of cycles per iteration, simply by indicating the situations that cause stall cycles. Calculate the CPI and CPI-useful.
- 4- For **SS2**, optimize the loop code using the "software pipeline" technique, clearly indicating the number of stages and which instructions belong to which stage. Then reorder to avoid lost cycles as far as possible. Use a simplified diagram to show how the loop is executed. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.

In the rest of the exercise, we focus on the **Mips** implementation.

The processor is connected to memory via a data cache and an instruction cache. The instruction cache is assumed to be perfect (Miss rate = 0). Several types of data cache are available. We wish to study the influence of the data cache on execution performance.

At the entry of the loop, the data cache is assumed to be empty. The color image contains 1 Kpixels and its address is aligned (multiple of 4096). The images resulting from the decomposition are placed at consecutive memory addresses. The address of the red image, **red_img**, is a multiple of 4096. It contains 1 Kbyte. The green image follows the red image, and the blue image follows the green image. In other words :

```
grn_img = red_img + 1024
blu_img = grn_img + 1024
```

1st case :

The data cache is a direct mapping Write Through cache. It has a capacity of 4 Kbytes. A cache block contains 16 bytes. In case of a hit, it takes 1 cycle to access the cache. In case of a Miss, it takes an average of 11 cycles to receive the missing block and store it into the cache memories. It then takes a further 1 cycle to respond to the processor.

First, we will just evaluate the effect of loads, ignoring the effect of stores.

- 5- How many Misses occur when executing the original code.
- 6- Calculate the average number of cycles per iteration. Calculate the CPI-useful of the loop.

We are now interested in the effect of stores.

The cache has a one-place write buffer. For a store operation, when the buffer is empty, it takes 1 cycle to write into the buffer. The processor can then continue executing instructions, and the cache takes over the actual writing to memory. On average, it takes 6 cycles to achieve the write and to empty the buffer. When the block to be modified is not present in the cache, it is not loaded into the cache (no Miss on stores).

- 7- Calculate the average number of cycles per iteration, taking into account the effect of loads and stores. Calculate the CPI-useful of the loop.
- 8- To improve the performance, we decide to unroll the loop 1 time (processing 2 pixels in each iteration). Optimize the resulted unrolled code, taking into account the data cache.
- 9- How many Misses occur during the execution of the iterations of the unrolled optimized code?
- 10- Calculate the average number of cycles per iteration of the unrolled code, taking into account the effect of loads and stores. Calculate the CPI-useful of the loop.
- 11- Would the Miss rate be different if, instead of a direct mapping cache, we used a 2-way associative cache ?

2nd case :

The data cache is a direct mapping Write Back cache. It has a capacity of 4 Kbytes. A cache block contains 16 bytes. On a store, when the block is not present in the cache, it is first loaded into the cache from memory and then modified. In the case of a Hit, it takes 1 cycle to access the cache. In the case of Miss, if the block to be replaced is Clean, it takes an average of 11 cycles to receive the missing block and to store it in the cache memories. If the block to be replaced is Dirty, it takes an average of 16 cycles to receive the missing block and to store it in the cache memories. It then takes a further 1 cycle to respond to the processor.

- 12- How many Misses (distinguish between 2 types of Miss) occur when executing iterations of the original code.
- 13- Calculate the average number of cycles per iteration. Calculate the CPI-useful of the loop.